AUTOMATED DATA CONTRACTS & ASSERTIONS

Dosen Pengampu: Pak Isbat Uzzin Nadhori S.Kom., M.T



Disusun untuk Memenuhi Tugas Mata Kuliah:

PRAKTIKUM DATA PROCESSING WORKFLOW

Oleh:

Udi Adhahaska Arta Ananta

3324600052

D4 Sains Data Terapan B

PROGRAM STUDI D4 SAINS DATA TERAPAN

DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

Link Github: https://github.com/Kyouji-Prinz/dpw-m8-data-contract.git

PRAKTIKUM DATA PROCESSING WORKFLOW

MODUL: AUTOMATED DATA CONTRACTS & ASSERTIONS (DATA CLEANING & TRANSFORMATION)

Langkah-langkah Praktikum:

1. Verifikasi Environment & Instalasi Library

Memeriksa kesiapan Python 3.12 serta memverifikasi instalasi pustaka utama: Pydantic v2, Pandas, dan Great Expectations (GX Core 1.x) untuk validasi kontrak data.

Perintah verifikasi terminal:

```
print(f"[OK] Pydantic version : {pydantic.__version__}")
print(f"[OK] Pandas version   : {pd.__version__}")
print(f"[OK] Great Exp version: {gx.__version__}")
```

```
[OK] Pydantic version : 2.14.0
[OK] Pandas version   : 3.0.3
[OK] Great Exp version: 1.24.0
```

Hasil: Sistem Python 3.12.10 aktif dengan dependensi pydantic v2.14.0, pandas v3.0.3, dan great_expectations v1.24.0 terpasang sempurna tanpa konflik dependensi.

2. Pembuatan Dataset Transaksi (Valid, Invalid, & Schema Broken)

Menyiapkan dataset batch transaksi e-commerce (Order Service) di dalam folder data/ untuk merepresentasikan skenario produksi:

• data/orders_valid.csv : 4 transaksi normal yang memenuhi seluruh aturan kontrak.

• data/orders_invalid.csv : 5 transaksi anomali (customer_id negatif, amount negatif, status tidak dikenal, dan status PAID tanpa tanggal bayar).

• data/orders_schema_broken.csv : 2 transaksi yang disusupi kolom liar (unexpected_field) untuk menguji proteksi kebocoran skema.

Perintah pembuatan dataset:

```
mkdir data src
```

```
# Menyimpan file orders_valid.csv, orders_invalid.csv, dan orders_schema_broken.csv ke folder data/
```

[Tempat Screenshot Output: Struktur File Dataset dan Tampilan Isi CSV]

Hasil: Dataset berhasil dibuat. Tabel anomali pada orders_invalid.csv:

Record ID	Field Bermasalah	Nilai Aktual	Deskripsi Pelanggaran Kontrak
ORD-101	Semua	201, 250000, PAID	Valid (Memenuhi seluruh kriteria)
ORD-102	customer_id	-202	Nilai bilangan negatif, melanggar constraint gt=0
ORD-103	amount	-75000.0	Nominal transaksi negatif, melanggar constraint ge=0
ORD-104	status	'UNKNOWN'	Status di luar domain nilai yang diizinkan (Literal)
ORD-105	payment_date	None / Kosong	Pelanggaran business rule: status PAID mewajibkan tanggal bayar

3. Pemahaman Konsep Data Contract & Pembahasan Pertanyaan Kunci

Data contract mendefinisikan batas kesepakatan formal antara producer (Order Service) dan consumer (Data Warehouse). Spesifikasi kontrak yang disepakati:

Field	Required	Tipe Data	Constraint / Aturan Bisnis
order_id	Ya	string	min_length=1, tidak boleh string kosong (Identifier)
customer_id	Ya	integer	> 0 (harus bilangan bulat positif)
amount	Ya	float	>= 0.0 (tidak boleh bernilai negatif)
status	Ya	enum / Literal	Hanya boleh bernilai 'PAID', 'PENDING', atau 'CANCELLED'
transaction_date	Ya	datetime	Valid ISO format datetime
payment_date	Tidak	datetime / null	Wajib terisi apabila status == 'PAID'

Pembahasan 7 Pertanyaan Evaluasi Percobaan 2:

1. Field Identifier: order_id merupakan pengenal unik (primary identifier) untuk setiap transaksi pesanan.

2. Field Wajib Memiliki Nilai: order_id, customer_id, amount, status, dan transaction_date wajib bernilai pada setiap baris data.

3. Field dengan Batasan Range: customer_id dibatasi bernilai > 0 (gt=0) dan amount dibatasi bernilai >= 0.0 (ge=0.0).

4. Field dengan Allowed Values: status memiliki domain terbatas berupa Literal['PAID', 'PENDING', 'CANCELLED'].

5. Aturan Business Rule: Aturan relasi kondisional antar-kolom di mana transaksi dengan status PAID mewajibkan ketersediaan payment_date.

6. Aksi Saat Satu Record Gagal: Pada tingkat row-level, dicatat pesan ValidationError dan baris tersebut dipisahkan ke dead-letter queue untuk diaudit.

7. Aksi Saat Satu Batch Gagal: Pada arsitektur Fail-Fast, seluruh batch langsung ditolak (reject) dan proses penulisan ke Data Warehouse dihentikan total.

Analisis Penggunaan extra='forbid' sebagai Elemen Kontrak Data:

Konfigurasi ConfigDict(extra='forbid') menjamin produsen data tidak dapat menyusupkan kolom liar tanpa kesepakatan. Dalam data pipeline skala besar, keberadaan kolom liar dapat merusak skema tabel downstream (schema drift), membebani storage, atau membuka kerentanan keamanan. Kebijakan ini memaksa produsen mengajukan pembaruan kontrak resmi sebelum menambah atribut baru.

4. Implementasi Data Contract dengan Pydantic v2 (src/contract.py)

Membangun model validasi berbasis Pydantic v2 dengan mode ketat (strict=True), larangan field ekstra (extra='forbid'), dan decorator custom validator:

```python
from datetime import datetime
from typing import Literal
from pydantic import BaseModel, ConfigDict, Field, ValidationError, model_validator

class OrderContract(BaseModel):
    model_config = ConfigDict(
        strict=True,
        extra="forbid",
    )

    order_id: str = Field(min_length=1)
    customer_id: int = Field(gt=0)
    amount: float = Field(ge=0)
    status: Literal["PAID", "PENDING", "CANCELLED"]
    transaction_date: datetime
    payment_date: datetime | None = None

    @model_validator(mode="after")
    def paid_requires_payment_date(self):
        if self.status == "PAID" and self.payment_date is None:
            raise ValueError("payment_date wajib diisi jika status=PAID")
        return self
```

```
[INFO] File 'contract.py' adalah modul definisi skema (OrderContract).
[OK] Skema Pydantic berhasil dimuat tanpa error!
```

Hasil: Model OrderContract siap digunakan sebagai validator inti baik untuk inspeksi satu baris maupun batch.

5. Validasi Satu Record Valid (Percobaan 4)

Menguji validasi data record ORD-001 yang mematuhi seluruh spesifikasi kontrak:

```python
def run_experiment_4():
    print("=" * 60)
    print("PERCOBAAN 4: Validasi Satu Record Valid")
    print("=" * 60)
    valid_order = {
        "order_id": "ORD-001",
        "customer_id": 101,
        "amount": 250000.0,
        "status": "PAID",
        "transaction_date": datetime(2026, 9, 20, 8, 0),
        "payment_date": datetime(2026, 9, 20, 8, 15),
    }
```

```
PERCOBAAN 4: Validasi Satu Record Valid
============================================================
STATUS: VALID
Data Terverifikasi: order_id='ORD-001' customer_id=101 amount=250000.0 status='PAI
D' transaction_date=datetime.datetime(2026, 9, 20, 8, 0) payment_date=datetime.dat
etime(2026, 9, 20, 8, 15)
```

Hasil: Pydantic berhasil memvalidasi dan mem-parsing data transaksi menjadi objek OrderContract yang terverifikasi.

6. Pengujian Data Invalid: Constraint Nilai Negatif (Percobaan 5)

Menguji keandalan assertion saat customer_id diubah menjadi bernilai negatif (-101):

```python
def run_experiment_5():
    print("\n" + "=" * 60)
    print("PERCOBAAN 5: Menguji Data Invalid (customer_id negatif: -101)")
    print("=" * 60)
    invalid_order = {
        "order_id": "ORD-001",
        "customer_id": -101,  # melanggar gt=0
        "amount": 250000.0,
        "status": "PAID",
        "transaction_date": datetime(2026, 9, 20, 8, 0),
        "payment_date": datetime(2026, 9, 20, 8, 15),
    }

    try:
        order = OrderContract.model_validate(invalid_order)
        print("STATUS: VALID")
        print(order)
    except ValidationError as exc:
        print("STATUS: INVALID")
        print(exc)
```

```
STATUS: INVALID
1 validation error for OrderContract
customer_id
  Input should be greater than 0 [type=greater_than, input_value=-101, input_type=
int]
    For further information visit https://errors.pydantic.dev/2.14/v/greater_than
```

Hasil: Pydantic secara akurat menolak record dengan melempar ValidationError bertipe 'greater_than' karena melanggar aturan Field(gt=0).

7. Pengujian Custom Business Rule: Status PAID vs Payment Date (Percobaan 6)

Menguji transaksi ORD-999 dengan status 'PAID' namun field payment_date bernilai None:

```python
def run_experiment_6():
    print("\n" + "=" * 60)
    print("PERCOBAAN 6: Menguji Business Rule (status=PAID, payment_date=None)")
    print("=" * 60)
    invalid_order = {
        "order_id": "ORD-999",
        "customer_id": 999,
        "amount": 100000.0,
        "status": "PAID",
        "transaction_date": datetime(2026, 9, 20, 10, 0),
        "payment_date": None,  # melanggar business rule model_validator
    }
```

```
STATUS: INVALID
1 validation error for OrderContract
  Value error, payment_date wajib diisi jika status=PAID [type=value_error, input_
value={'order_id': 'ORD-999', '...), 'payment_date': None}, input_type=dict]
    For further information visit https://errors.pydantic.dev/2.14/v/value_error
```

Hasil: Validator @model_validator(mode='after') mendeteksi ketidaksesuaian relasi data dan menolak transaksi dengan pesan eksplisit.

8. Pengujian Extra Field: Larangan Atribut Liar (Percobaan 7)

Menguji proteksi terhadap penyusupan atribut asing ('unexpected_field': 'ABC'):

```python
def run_experiment_7():
    print("\n" + "=" * 60)
    print("PERCOBAAN 7: Menguji Extra Field (extra='forbid')")
    print("=" * 60)
    invalid_order = {
        "order_id": "ORD-888",
        "customer_id": 888,
        "amount": 100000.0,
        "status": "PENDING",
        "transaction_date": datetime(2026, 9, 20, 10, 0),
        "payment_date": None,
        "unexpected_field": "ABC",  # extra field
    }
```

```
============================================================
STATUS: INVALID
1 validation error for OrderContract
unexpected_field
  Extra inputs are not permitted [type=extra_forbidden, input_value='ABC', input_t
ype=str]
    For further information visit https://errors.pydantic.dev/2.14/v/extra_forbidd
en
```

Hasil: Parameter extra='forbid' menggagalkan proses validasi, membuktikan bahwa skema pipeline terkunci rapat dari kolom liar.

9. Validasi Batch Data Valid (Percobaan 8)

Menjalankan validasi terhadap seluruh kumpulan baris pada dataset orders_valid.csv:

```
[PIPELINE START] Membaca dataset: ../data/orders_valid.csv
Jumlah record : 4
Valid         : 4
Invalid       : 0

================================================
BATCH VALID: Semua record memenuhi spesifikasi Data Contract.
SUCCESS: 4 record berhasil dimuat ke Data Warehouse (../data/warehouse_orders.csv)
.
================================================
```

Hasil: Seluruh baris data lolos verifikasi kontrak tanpa ada kesalahan, batch dialirkan ke tahap penyimpanan.

10. Validasi Batch Data Invalid & Ekstraksi Error Detail (Percobaan 9)

Menjalankan validasi batch pada dataset anomali orders_invalid.csv:

```
[PIPELINE START] Membaca dataset: ../data/orders_invalid.csv
Jumlah record : 5
Valid         : 1
Invalid       : 4

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
FAIL-FAST: BATCH DITOLAK KARENA MEMILIKI DATA CACAT!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Detail Pelanggaran Data Contract:
 -> Baris 3 (order_id=ORD-102):
    - Lokasi: 'customer_id' | Error: Input should be greater than 0 (greater_than)
 -> Baris 4 (order_id=ORD-103):
    - Lokasi: 'amount' | Error: Input should be greater than or equal to 0 (greate
r_than_equal)
 -> Baris 5 (order_id=ORD-104):
    - Lokasi: 'status' | Error: Input should be 'PAID', 'PENDING' or 'CANCELLED' (
literal_error)
 -> Baris 6 (order_id=ORD-105):
    - Lokasi: '' | Error: Value error, payment_date wajib diisi jika status=PAID (
value_error)

[QUALITY GATE REJECT] Proses dihentikan sebelum menyentuh Data Warehouse.
```

Hasil: Sistem berhasil mengidentifikasi 4 transaksi cacat dengan menyajikan nomor baris, order_id, dan aturan spesifik yang dilanggar.

11. Penerapan Gerbang Kualitas (Quality Gate) & Fail-Fast (Percobaan 10)

Mekanisme gerbang kualitas Fail-Fast diterapkan melalui pengujian kondisi: jika daftar error tidak kosong, sistem langsung menghentikan proses eksekusi (raise SystemExit(1)) sebelum fungsi save_to_warehouse() dipanggil.

```
if errors:
        print("\n" + "!" * 50)
        print("FAIL-FAST: BATCH DITOLAK KARENA MEMILIKI DATA CACAT!")
        print("!" * 50)
        print("Detail Pelanggaran Data Contract:")
        for err in errors:
            print(f" -> Baris {err['row']} (order_id={err['order_id']}):")
            for sub_err in err["errors"]:
                loc = ".".join(str(l) for l in sub_err.get("loc", []))
                msg = sub_err.get("msg", "")
                err_type = sub_err.get("type", "")
                print(f"    - Lokasi: '{loc}' | Error: {msg} ({err_type})")

        if fail_fast:
            print("\n[QUALITY GATE REJECT] Proses dihentikan sebelum menyentuh Data Warehouse.")
            return False, valid_orders, errors

    print("\n" + "=" * 50)
    print("BATCH VALID: Semua record memenuhi spesifikasi Data Contract.")
    save_to_warehouse(valid_orders, warehouse_path)
    print(f"SUCCESS: {len(valid_orders)} record berhasil dimuat ke Data Warehouse ({warehouse_path}).")
    print("=" * 50)
    return True, valid_orders, errors


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Automated Data Contract Batch Validator")
    parser.add_argument("--data", default="../data/orders_valid.csv", help="Path ke file dataset CSV")
    parser.add_argument("--warehouse", default="../data/warehouse_orders.csv", help="Path file Data Warehouse target")
    parser.add_argument("--no-fail-fast", action="store_true", help="Nonaktifkan exit code saat gagal")

    args = parser.parse_args()

    success, valids, errs = run_pipeline(
        input_path=args.data,
        warehouse_path=args.warehouse,
        fail_fast=True
    )

    if not success and not args.no_fail_fast:
        raise SystemExit(1)
```

[Tempat Screenshot Output: Bukti Exit Code 1 / Fail-Fast di Terminal]

Hasil: Eksekusi berhenti seketika dengan status exit code 1. Data cacat tidak diberikan toleransi untuk diproses lebih lanjut.

12. Simulasi Pemuatan ke Data Warehouse Target (Percobaan 11)

Data Warehouse disimulasikan sebagai file data/warehouse_orders.csv yang hanya menerima transaksi yang telah lolos validasi kontrak secara atomik:

```python
def save_to_warehouse(orders, output_path):
    os.makedirs(os.path.dirname(os.path.abspath(output_path)), exist_ok=True)
    with open(output_path, "w", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        writer.writerow(
            [
                "order_id",
                "customer_id",
                "amount",
                "status",
                "transaction_date",
                "payment_date",
            ]
        )
        for order in orders:
            writer.writerow(
                [
                    order.order_id,
                    order.customer_id,
                    order.amount,
                    order.status,
                    order.transaction_date.isoformat(),
                    order.payment_date.isoformat() if order.payment_date else "",
                ]
            )
```

	A	B	C	D	E	F
1	orde...	custom...	amo...	status	transaction_date	payment_date
2	ORD...	101	2500...	PAID	2026-09-20T08:0...	2026-09-20T08:1...
3	ORD...	102	1250...	PEN...	2026-09-20T08:0...	
4	ORD...	103	7500...	CAN...	2026-09-20T08:1...	
5	ORD...	104	5000...	PAID	2026-09-20T08:2...	2026-09-20T08:3...

Hasil: Tabel Data Warehouse terisi 4 record transaksi dari batch valid pertama.

13. Pembuktian Empiris Mekanisme Fail-Fast (Percobaan 12)

Uji pembuktian dilakukan dengan menjalankan pipeline berturut-turut:

1. Eksekusi orders_valid.csv -> Hasil: BATCH VALID, 4 transaksi masuk ke warehouse_orders.csv.
2. Eksekusi orders_invalid.csv -> Hasil: FAIL-FAST: BATCH DITOLAK.

3. Memeriksa kembali isi warehouse_orders.csv:

```python
context = gx.get_context(mode='ephemeral')
ds = context.data_sources.add_pandas('gx_source')
asset = ds.add_dataframe_asset('gx_asset')
batch_def = asset.add_batch_definition_whole_dataframe('gx_batch')

suite = gx.ExpectationSuite(name='order_suite')
suite.add_expectation(gxe.ExpectColumnValuesToNotBeNull(column='order_id'))
suite.add_expectation(gxe.ExpectColumnValuesToBeBetween(column='customer_id', min_value=1))
suite.add_expectation(gxe.ExpectColumnValuesToBeBetween(column='amount', min_value=0.0))
suite.add_expectation(gxe.ExpectColumnValuesToBeInSet(column='status', value_set=['PAID', 'PENDING', 'CANCELLED']))
suite.add_expectation(gxe.ExpectColumnValuesToNotBeNull(column='transaction_date'))
context.suites.add(suite)

val_def = gx.ValidationDefinition(name='order_val_def', data=batch_def, suite=suite)
context.validation_definitions.add(val_def)

res_valid = val_def.run(batch_parameters={'dataframe': df_valid})
res_invalid = val_def.run(batch_parameters={'dataframe': df_invalid})
```

```
ype ..\data\warehouse_orders.csv
order_id,customer_id,amount,status,transaction_date,payment_date
ORD-001,101,250000.0,PAID,2026-09-20T08:00:00,2026-09-20T08:15:00
ORD-002,102,125000.0,PENDING,2026-09-20T08:05:00,
ORD-003,103,75000.0,CANCELLED,2026-09-20T08:10:00,
ORD-004,104,500000.0,PAID,2026-09-20T08:20:00,2026-09-20T08:30:00
```

Hasil: Terbukti secara empiris file warehouse_orders.csv tetap hanya berisi 4 data valid awal. Data rusak dari batch kedua tidak pernah masuk ke warehouse, membuktikan kesuksesan Quality Gate Fail-Fast.

14. Validasi Dataset-Level Menggunakan Great Expectations (GX Core 1.x)

Melengkapi validasi baris Pydantic dengan pengujian tingkat dataset menggunakan Great Expectations (src/validate_gx.py) untuk mengevaluasi dataset secara deklaratif:



```python
def build_gx_pipeline():
    context = gx.get_context(mode="ephemeral")

    # 1. Menyiapkan Data Source & Data Asset untuk Pandas DataFrame
    data_source = context.data_sources.add_pandas("orders_datasource")
    data_asset = data_source.add_dataframe_asset(name="orders_asset")
    batch_definition = data_asset.add_batch_definition_whole_dataframe("orders_batch_def")

    # 2. Merancang Expectation Suite sesuai Kontrak Data
    suite = gx.ExpectationSuite(name="orders_data_contract_suite")

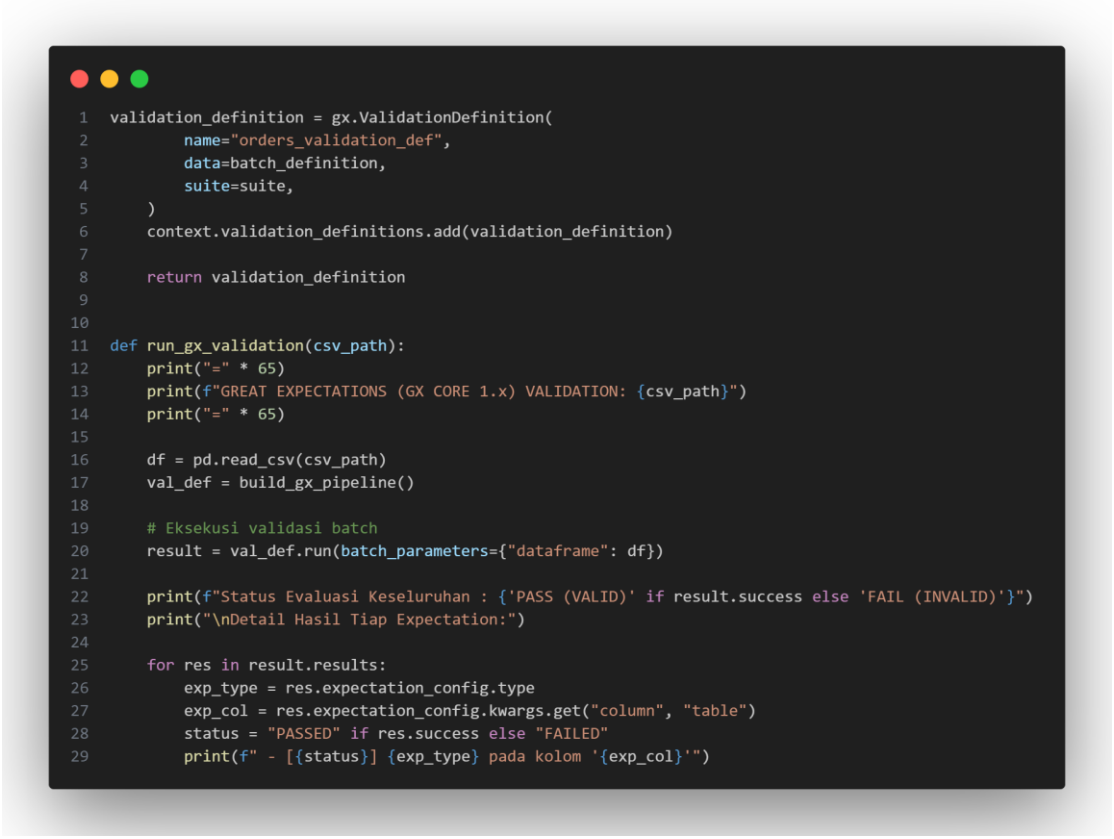
    # Rule 1: order_id tidak boleh null
    suite.add_expectation(gxe.ExpectColumnValuesToNotBeNull(column="order_id"))

    # Rule 2: customer_id harus positif (> 0)
    suite.add_expectation(gxe.ExpectColumnValuesToBeBetween(column="customer_id", min_value=1))

    # Rule 3: amount harus non-negatif (>= 0.0)
    suite.add_expectation(gxe.ExpectColumnValuesToBeBetween(column="amount", min_value=0.0))

    # Rule 4: status harus masuk dalam set allowed values
    suite.add_expectation(
        gxe.ExpectColumnValuesToBeInSet(
            column="status",
            value_set=["PAID", "PENDING", "CANCELLED"]
        )
    )

    # Rule 5: transaction_date tidak boleh null
    suite.add_expectation(gxe.ExpectColumnValuesToNotBeNull(column="transaction_date"))
```



```python
validation_definition = gx.ValidationDefinition(
        name="orders_validation_def",
        data=batch_definition,
        suite=suite,
    )
    context.validation_definitions.add(validation_definition)

    return validation_definition


def run_gx_validation(csv_path):
    print("=" * 65)
    print(f"GREAT EXPECTATIONS (GX CORE 1.x) VALIDATION: {csv_path}")
    print("=" * 65)

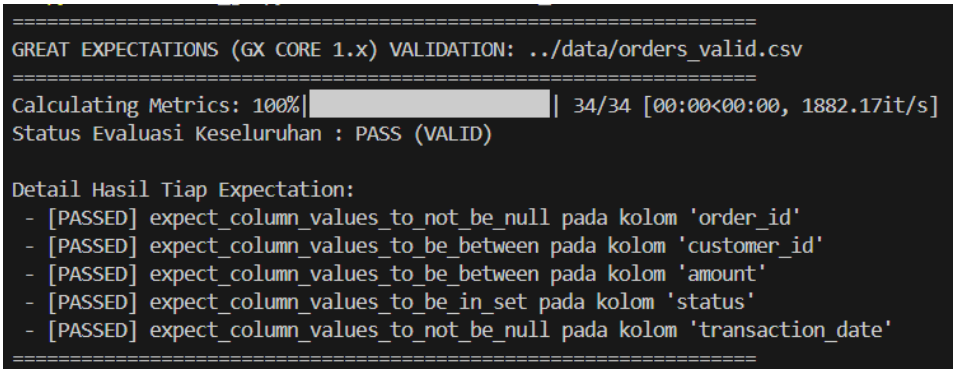
    df = pd.read_csv(csv_path)
    val_def = build_gx_pipeline()

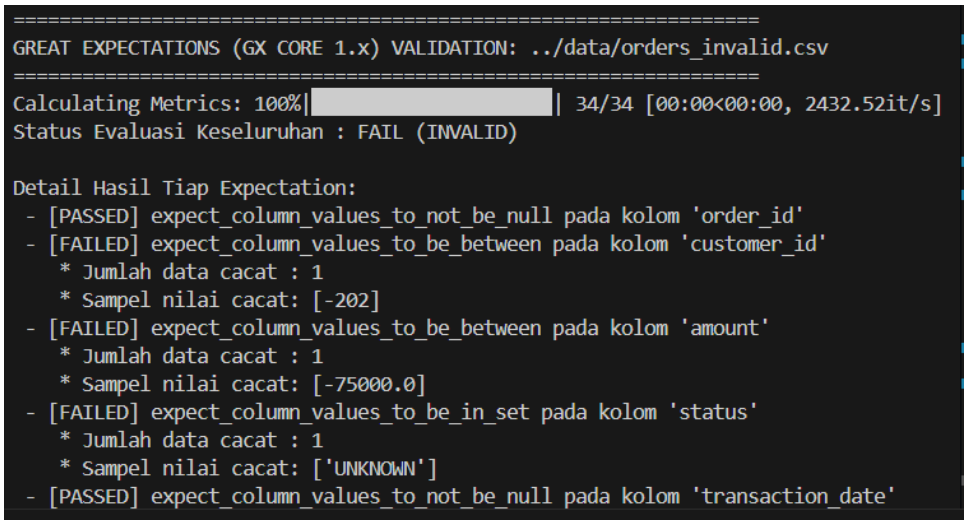
    # Eksekusi validasi batch
    result = val_def.run(batch_parameters={"dataframe": df})

    print(f"Status Evaluasi Keseluruhan : {'PASS (VALID)' if result.success else 'FAIL (INVALID)'}")
    print("\nDetail Hasil Tiap Expectation:")

    for res in result.results:
        exp_type = res.expectation_config.type
        exp_col = res.expectation_config.kwargs.get("column", "table")
        status = "PASSED" if res.success else "FAILED"
        print(f" - [{status}] {exp_type} pada kolom '{exp_col}'")
```

```
python validate_gx.py --data ../data/orders_valid.csv
python validate_gx.py --data ../data/orders_invalid.csv
```





Hasil: Great Expectations berhasil memverifikasi dataset valid dengan status PASS dan mendeteksi anomali kolom pada dataset invalid secara deklaratif.

Evaluasi Akhir & Refleksi Teknis

1. Apa keuntungan menerapkan Data Contract pada Ingestion Boundary dibandingkan melakukan pembersihan data di Data Warehouse?

Pembersihan data di Data Warehouse bersifat reaktif dan mahal secara komputasi. Dengan menerapkan Data Contract pada ingestion boundary (pintu masuk), data cacat langsung dicegah sebelum masuk ke sistem penyimpanan. Hal ini menghemat biaya komputasi ETL/ELT downstream, mencegah rusaknya dashboard analitik analitis, dan menjaga integritas data hulu ke hilir.

2. Mengapa pendekatan Fail-Fast sangat penting dalam arsitektur Data Engineering modern?

Prinsip Fail-Fast memastikan kegagalan terdeteksi seawal mungkin sebelum efek kerusakan merembes ke tahap downstream yang lebih kompleks. Bila transaksi batch bersifat atomik, membiarkan sebagian data cacat lolos akan menghasilkan anomali agregasi finansial yang sulit dilacak. Menolak batch secara utuh memaksa produsen segera memperbaiki anomali pada sumbernya.

3. Apa perbedaan peran mendasar antara Pydantic dan Great Expectations?

Tabel Komparasi Menyeluruh Pydantic v2 vs Great Expectations:

Dimensi Perbandingan	Pydantic v2	Great Expectations (GX Core 1.x)
Tingkat Granularitas	Row-level (objek per baris data) & type coercion	Dataset / Batch-level (agregasi tabular & profiling)
Business Rule Antar-kolom	Sangat fleksibel via Python decorator (@model_validator)	Berbasis ekspresi SQL / rule lintas kolom dataset
Kecepatan & Performa	Ultra-cepat (pydantic-core Rust), latency mikrodetik	Komprehensif dengan profiling statistik dan validasi metrik
Posisi Arsitektur Ideal	Gerbang pertama (Front Gate) saat ingest data masuk	Gerbang downstream untuk audit kualitas tabel Data Warehouse / Lake

Refleksi Teknis & Kesimpulan Praktikum:

Keputusan arsitektur terbaik dalam pipeline pemrosesan data adalah menggabungkan kedua teknologi tersebut dalam arsitektur pertahanan berlapis (defense in depth):

1. Pydantic v2 bertindak sebagai first line of defense pada Producer Service / API Gateway untuk memvalidasi tipe ketat, ketiadaan extra fields liar, serta aturan bisnis spesifik.

2. Great Expectations bertindak sebagai second line of defense pada data pipeline sebelum tahap finalisasi Data Warehouse untuk memverifikasi keunikan, completeness, serta profil distribusi statistik secara menyeluruh.